

Technical Assessment {.NET Developer}

Overview	1
Challenge	1
Technical Requirements	2
Functional Requirements	2
Bonus (Optional)	3
Considerations	4
Deliverables	4

Overview

- This technical assessment is to demonstrate your proficiency in ASP.NET Core MVC and Razor to create a full-stack web application.
- The ideal applicant should possess a good understanding of ASP.NET Core MVC, Entity Framework Core, database design, and server-rendered front-end development with Razor views, partial views, and view components. Use any UI or JavaScript libraries you desire; do not use a single-page application framework.
- We ask that you complete everything in the requirements section. Items outlined in the “bonus” section are completely optional, and your evaluation will not be negatively affected if you choose not to complete them.

Challenge

Create a full-stack web application for a property management company that accepts rental applications for its apartments. A property manager maintains properties and their units. An applicant applies for a unit and, if approved, is issued a lease. There are only two roles in this system: an applicant and a property manager. There are certain actions that an applicant and a property manager can perform; ensure the permissions in the controllers and the UI reflect this.

- a. The applicant can create, fill out, submit, and withdraw applications for available units, and correct and resubmit an application that was returned.
- b. The property manager maintains properties and units and reviews submitted applications. An approved application issues a twelve-month lease for the unit.

Technical Requirements

1. Build the web application with ASP.NET Core MVC and Razor:
 - a. Use controllers, view models, Razor views, partial views, and view components. Both partial views and view components are required.
 - b. Populate modals from partial views returned by controller actions. When the submitted form fails validation, return the same partial with the validation messages so the modal re-renders in place; when it succeeds, close the modal and refresh the affected part of the page.
 - c. Structure and style the web application to your liking using best practices.
2. Create the backend using .NET 10:
 - a. Use ASP.NET Identity for User/Role Management.
 - b. On Start:
 - i. The database should be created, and migrations applied.
 - ii. The database should be seeded idempotently with lookups, property managers, applicants, properties, units, and applications in every status.
 1. Seed your database with data using Bogus for .NET.
 - c. Add unit tests for business logic.
3. Use SQL Server / SQL Server Express:
 - a. Use Entity Framework Core with code-first database migrations.

Functional Requirements

1. Users:
 - a. Users should be able to sign up, log in, and log out.
 - i. For convenience, sign-up allows the user to specify if they are an Applicant or a Property Manager.
2. Properties and Units:
 - b. A property manager can add, edit, and remove properties and their units (unit number, bedrooms, monthly rent, unit type) through modals. Applicants can browse available units and start an application for one.
 - c. Unit Type is a lookup with Active and Inactive values. An inactive value still displays on a unit that already uses it but cannot be selected for any other unit. Enforce this on the server.
 - d. Approval creates a lease for the unit with a start date and a twelve-month term. A unit whose lease term covers today is not available.
4. Rental Application
 - a. An application is for one unit and has two sections and a summary:
 - i. Applicant Information (name, phone, email, current address)
 - ii. Residence History (a list of prior residences, each with an address, landlord name and phone, and move-in and move-out dates)

- iii. Summary (read-only view of both sections and Submit)
 - b. The application is a single page that shows one section at a time, not a separate page per section. One view model drives it, and each section renders through its own partial view or view component. One form posts to one action, and the button clicked determines what happens:
 - i. Continue validates the current section, persists it only when it is valid, and shows the next section (or the Summary after the last one). When the section is invalid it re-renders with the errors.
 - ii. Back returns to the previous section without saving. Submit is available from the Summary only once both sections have been saved.
 - c. Residences are added, edited, and removed through a modal.
 - d. The same section partial renders editable or read-only based on a server-side decision. An applicant can edit while the application is in a Draft or Returned status; otherwise, every section is read-only. Controllers reject posts that are not allowed.
 - e. At submit and again at approval, reject the action with an error when the unit has an active lease. Other open applications for that unit are left as they are. The approval check prevents a second lease.
- 5. Review
 - a. A property manager opens a submitted application and completes it through a review modal with an outcome of Approve, Return, or Deny and a comment (required for Return and Deny).
 - b. Application statuses are Draft, Submitted, Returned, Approved, Denied, and Withdrawn. Approved, Denied, and Withdrawn are terminal.
 - c. The application page shows property managers a history of status changes and review outcomes (who, when, comment).
- 6. Application List
 - a. A list of applications filtered by status and property, with the filtering done in the database, not in memory. Applicants see their own applications and property managers see all of them.

Bonus (Optional)

1. Add paging and sorting to the application list, done in the database, then extract the list into a reusable grid view component driven by a JSON endpoint that returns the page of rows and the filtered total, documented with OpenAPI
2. Add a review queue. A property manager claims a submitted application (Under Review) before completing it, and can release it back to the queue.
3. Add property manager notes that are visible and editable only to property managers and are never rendered or returned to an applicant



4. Allow a section to be saved even when it fails validation, showing the errors. The Summary lists everything still blocking submission, and Submit stays blocked while any error remains. Define rules once per section and return errors to the field they belong to.
5. Allow more than one applicant on an application. Any applicant on it can view and edit it. Ownership checks apply to all of them.
 - a. With two applicants editing at once, saves to different sections must not interfere with each other, and when both save the same section, the second save is rejected as stale with a message to reload rather than silently overwriting. No real-time synchronisation is expected.

Considerations

- Any elements not specifically detailed in the guidelines are left to your discretion, and you are encouraged to use your sound judgment in completing the assessment as you see fit.
- Approach this exercise like any other development task, ensuring the code you produce is clean, robust, and ready for a production environment.
- You should follow the standards, conventions, and best practices associated with any frameworks that you use.
- It's important to integrate thoughtful design principles into your work. Be ready to explain the rationale behind your design decisions and the specifics of your implementations.
- The exercise is not timed, and you are not rewarded for a fast turnaround; however, **we ask that you return it within (7) days from today's date**. If unforeseen situations demand more time, please let us know your requested delivery date. Failure to deliver on the requested date may result in disqualification.

Deliverables

- When complete, respond to this email with a link to your GitHub repository.
- Please ensure that your project includes a README.md file, containing any necessary setup or installation instructions that are not standard for the technology used in this assessment.